



PRÁCTICA 2 - PROCESADORES CMP: USANDO LA VECTORIZACIÓN Y LA PARALELIZACIÓN

Asignatura: ARQUITECTURA Y ORGANIZACIÓN DE COMPUTADORES

Alumno: ALBERTO DOMINGO LÓPEZ (a.domingolopez@um.es)

Grupo y Subgrupo: 2.1

Curso: 2025/2026

1. Características del computador.

Reutilizamos la tabla de las características del computador de la primera práctica, puesto que tendremos que tener en cuenta algunas de ellas a la hora de entender y explicar aspectos de vectorización y paralelización.

NOMBRE	Alberto
SISTEMA OPERATIVO	Ubuntu 24.04.3 LTS
	CPU
MODELO	Intel Core i7-11800H
MICROARQUITECTURA	Tiger Lake
NÚMERO DE CORES	8
SMT/HyperThreading	Si
FRECUENCIA MÍNIMA	800 MHz
FRECUENCIA MÁXIMA	4.60 GHz
FECHA DE LANZAMIENTO	Primer trimestre 2021
LITOGRAFÍA	10 nm
TDP	45(up) – 35(down)
CACHÉ L1 INSTRUCCIONES	48 KiB (64 conjuntos, 12 vías, 64 bytes/bloque) por core (privada)
CACHÉ L1 DATOS	32 KiB (64 conjuntos, 8 vías, 64 bytes/bloque) por core (privada)
CACHÉ L2	1.3MB por core (1024 conjuntos, 20 vías, 64 bytes/bloque, privada)
CACHÉ L3	24MB (32768 conjuntos, 12 vías, 64 bytes/bloque, compartida)
EXTENSIONES ISA SOPORTADAS	SSE4.1/4.2, AVX2, AVX-512
TAMAÑO DE REGISTROS VECTORIALES	512 bits (16 floats o 8 doubles) AVX-512
UNIDADES FPU (vectoriales)	2 Add, 2 Mul
	MEMORIA
TOTAL INSTALADA	16 GiB DDR4
CONTROLADORES	1
CANALES/CONTROLADOR	2
FRECUENCIA	3200 MHz

2. Benchmark *test_vectorization.cpp*.

Este benchmark mide el rendimiento obtenido a la hora de sumar un pequeño array de números en coma flotante de simple precisión. Concretamente hay cuatro implementaciones diferentes:

- *sum_vectors_vectorized*: un simple for, vectorización únicamente por parte de los compiladores.
- *sum_vectors_no_vectorized*: misma que la anterior, pero se le indica a los compiladores que no hagan ningún intento de vectorización.

- `sum_vectors_vectorized_restrict`: ahora se le indica que no hay aliasing usando `__restrict__`.
- `sum_vectors_vectorized_aligned`: le indicamos al compilador que los datos están alineados en memoria con la función `std::assume_aligned`.

Resultados:

	no-vectorized	vectorized	vectorized-restrict	vectorized-aligned
GCC	3.05±0.00	18.50±0.0	17.59±0.01	18.49±0.03
ICC	3.34±0.00	15.20±2.38	16.35±1.84	16.75±1.71
Clang	3.41±0.01	19.83±0.05	19.92±0.05	19.66±0.05

**Los resultados de las celdas son GFLOPS*

Como podemos observar, la vectorización influye de manera muy significativa en el rendimiento del programa. En los tres compiladores comprobamos que la versión no vectorizada no supera los 3 GFLOPS, mientras que aplicando algún tipo de vectorización, su rendimiento se multiplica por 5 (ICC) o 6 (GCC y Clang).

Si comparamos los compiladores, se aprecia que Clang ofrece ligeramente un mejor rendimiento global, seguido de GCC y finalmente ICC. Este resultado puede explicarse porque Clang emplea heurísticas más agresivas de vectorización y optimización de bucles, mientras que GCC es un poco más conservador pero mantiene una ejecución más estable. Sin embargo, ICC puede verse limitado en esta imagen Docker, ya que podría no estar detectando correctamente la microarquitectura real del procesador o no habilitar las instrucciones AVX más recientes.

Y si comparamos los diferentes tipos de vectorización, no observamos diferencias excesivamente notables. En GCC funciona ligeramente peor la versión con `__restrict__`; en ICC si se aprecian mejoras entre la vectorización automática del compilador y las versiones con más indicaciones, siendo ligeramente mejor la versión con `aligned`; y en Clang es donde menos diferencia se aprecia en las tres opciones de vectorización, lo que nos sugiere que este compilador ya optimiza de forma interna la alineación y el aliasing de punteros de forma eficiente.

3. Benchmark *loops.cpp*.

Este programa declara tres arrays (A, B y C) de tamaño 1000000, y en el main se declara la variable *total* inicializada a 0, y un bucle externo que se repite 100 veces, que tiene en su interior 5 bucles (sin anidación) que realizan diferentes tipos de

operaciones aritméticas con los arrays, para actualizar en el bucle final la variable total y mostrarla finalmente por pantalla.

3.1. Dependencias.

Vamos a analizar cada bucle interno por separado para ver sus dependencias y tratar de explicar cómo afectarían a una posible vectorización automática:

- Bucle 1:

```
for (int i = 0; i < arrays_size - 1; ++i) {  
    A[i] = i * 2 / 3;  
    B[i + 1] = A[i] - B[i];  
}
```

A[i] no tiene ninguna dependencia entre iteraciones, mientras que B[i+1] si depende de B[i] (dependencia RAW), por lo que no sería posible la vectorización automática, ya que si el compilador procesa varias iteraciones de manera automática, cambiarían los resultados con respecto a la ejecución original del bucle.

- Bucle 2:

```
for (int i = 0; i < arrays_size - 1; ++i) {  
    A[i] = A[i] - B[i];  
    B[i + 1] = C[i] * 2;  
}
```

Aunque parece que A[i] puede tener dependencia, no la tiene realmente, ya que se lee y se escribe en la misma iteración. Sin embargo, en el array B si hay dependencia, concretamente de tipo WAR, ya que en la iteración 0 se escribe B[1] y en la iteración 1 se lee B[1] y así sucesivamente. Por lo tanto, también afectaría a la vectorización, ya que los resultados no serían los esperados.

- Bucle 3:

```
for (int i = arrays_size - 1; i > 0; --i) {  
    A[i - 1] = A[i] + C[i];  
    B[i] = C[i] - B[i];  
}
```

En este caso quien presenta dependencias es A, ya que en cada iteración se lee A[i] y se escribe A[i-1], existiendo una dependencia RAW, e impidiendo la vectorización del bucle, ya que se alterarían los resultados.

- **Bucle 4:**

```
for (int i = 0; i < arrays_size; ++i) {  
    C[i] = A[i] + 2 * B[i];  
}
```

En este bucle no hay ningún tipo de dependencia, ya que solo se escribe C y se leen A y B, por lo que es totalmente posible la vectorización, ya que los resultados no se alterarían.

- **Bucle 5:**

```
for (int i = 0; i < arrays_size; ++i) {  
    total = total + C[i];  
}
```

La variable *total* se actualiza en cada iteración, pero como necesita su valor previo, existe una dependencia de datos de tipo RAW, impidiendo la vectorización automática. Aunque este bucle tiene algo de especial, ya que se podría usar una técnica llamada reducción para poder aplicar vectorización.

3.2. Informe de optimización.

Cuando compilamos con *make* se genera el informe de optimización *loops-icc.optrpt*, que nos muestra qué bucles han sido vectorizados y cuáles no, además de mostrar información sobre alineación de datos, longitudes de vector, etc.

El compilador ICC ha analizado todos los bucles que componen el programa y ha intentado aplicar vectorización automática, pero solo ha sido posible en tres de ellos, ya que en los otros dos había dependencias. Vamos a ver qué ha hecho el compilador para cada bucle:

- **Bucle 1:** aquí tenemos dos comentarios del informe que nos dicen muy bien qué sucede. El primero es “*loop was not vectorized: vectorization possible but seems inefficient.*” Esto significa que al principio el compilador determina que técnicamente sería posible la vectorización, pero que no va a ser eficiente y por eso no lo hace. De hecho, nos muestra un speedup estimado de 0.150, lo que claramente indica que el coste estimado supera el beneficio estimado.

Y el segundo comentario, muy interesante, es el siguiente: “loop was not vectorized: vector dependence prevents vectorization; vector dependence: assumed FLOW dependence between B[i+1] (19:7) and B[i] (19:7); vector dependence: assumed ANTI dependence between B[i] (19:7) and B[i+1] (19:7)”. Esto confirma nuestro análisis del apartado anterior, existe una

dependencia de tipo RAW (marcada como FLOW en el informe), entre $B[i]$ y $B[i+1]$, ya que cada iteración necesita valores calculados en la anterior. Sin embargo, nos marca también una antidependencia, la cual no habíamos tenido en cuenta en el análisis anterior. Realmente esta dependencia va unida de la anterior, ya que se lee $B[i]$ y se escribe $B[i+1]$ en iteraciones cercanas, entonces el compilador detecta un posible solapamiento al vectorizar y marca esa dependencia.

Para que este bucle pueda ser vectorizado, es necesario aplicar algunas transformaciones manuales. Una manera sería usar un vector auxiliar:

```
for (int i = 0; i < arrays_size; ++i)
    A[i] = i * 2 / 3;

vector<double> aux(B, B + arrays_size);

for (int i = 0; i < arrays_size - 1; ++i)
    B[i + 1] = A[i] - aux[i];
```

De esta manera, ya no habría dependencias entre iteraciones, ya que $B[i+1]$ ya no depende de los valores recién escritos de B, sino del auxiliar. El inconveniente es la copia extra del array B, lo que podría reducir ligeramente la ganancia de rendimiento.

- **Bucle 2:** en este caso el informe nos dice que el bucle sí ha sido vectorizado, con un speedup estimado de 3.990. Además, aparece un mensaje bastante interesante al principio: *“Loop Statements Reordered”*. Esto quiere decir que el compilador ha reordenado las instrucciones dentro del cuerpo del bucle para evitar posibles dependencias entre iteraciones, pero sin alterar el resultado final.

En nuestro análisis teórico anterior, habíamos identificado una dependencia de tipo WAR en el array B. El compilador, sin embargo, ha sido capaz de reordenar las operaciones para eliminar cualquier riesgo asociado a esa dependencia y aplicar la vectorización de manera segura.

Por otro lado, el mensaje *“reference $B[i+1]$ has unaligned access”* indica simplemente que el acceso a esa dirección de memoria no está alineado, pero esto no impide la vectorización, ya que el compilador puede usar instrucciones de carga y almacenamiento no alineadas sin afectar al resultado final.

- **Bucle 3:** el informe nos dice que el bucle no ha sido vectorizado debido a dependencias de datos. En concreto nos marca dos: *“assumed FLOW”*

dependence between A[i-1] (28:7) and A[i] (28:7)” y “assumed ANTI dependence between A[i] (28:7) and A[i-1] (28:7)”.

En nuestro análisis teórico habíamos señalado que existía una dependencia tipo RAW porque en cada iteración se lee A[i] y se escribe A[i-1]. Es prácticamente el mismo caso que el primer bucle, por eso también se marca una dependencia de tipo WAR, ya que se escribe A[i] mientras que todavía hay que leer A[i]. Por tanto, está claro que si el compilador aplica vectorización, los resultados podrían cambiar, ya que se rompería la secuencia lógica de cálculos.

En cuanto a posibles transformaciones manuales, podríamos aplicar una estrategia parecida a la del primer bucle: utilizar un vector auxiliar para eliminar dependencias y que el compilador pueda vectorizar. Pero tendríamos el mismo problema de memoria, así que habría que analizar si el beneficio que obtenemos merece la pena. El bucle quedaría así:

```
vector<double> aux(arrays_size);

for (int i = arrays_size - 1; i > 0; --i) {
    aux[i - 1] = A[i] + C[i];
    B[i] = C[i] - B[i];
}

for (int i = 0; i < arrays_size - 1; ++i){
    A[i] = aux[i];
}
```

- **Bucle 4:** como ya habíamos predicho, este bucle sí ha sido vectorizado, ya que no presenta ninguna dependencia que pueda afectar al resultado final. Si miramos el speedup, nos da una estimación de 4.570, lo que nos indica que la vectorización no solo es posible, sino que resulta muy eficiente.

El informe también nos da información interesante sobre cómo ha trabajado el compilador. Por ejemplo, dice que la longitud vectorial es 4, lo que significa que usa registros vectoriales de 256 bits (4 * 64 bits de un double), propios de la arquitectura AVX2. También especifica que todas las referencias (A[i], B[i], C[i]) están alineadas en memoria, lo cual permite el uso de instrucciones SIMD alineadas, más rápidas que las no alineadas.

- **Bucle 5:** el informe nos indica que el bucle sí ha sido vectorizado, y confirma que ha usado la técnica de reducción, tal y como habíamos dicho antes, a

pesar de la dependencia de tipo RAW en la variable total. Este tipo de dependencia normalmente impide la vectorización automática, ya que cada iteración depende del valor acumulado en la anterior. Sin embargo, el compilador ha reconocido el patrón de acumulación y lo ha transformado automáticamente en una reducción paralela, realizando sumas parciales en diferentes registros vectoriales para combinar todos los resultados en un único valor total.

Podemos concluir que el compilador ICC ha aprovechado de forma eficiente las capacidades de la arquitectura AVX2, utilizando registros de 256 bits para ejecutar operaciones vectoriales sobre bloques de cuatro números en coma flotante de doble precisión. En los bucles sin dependencias o con dependencias que se pueden resolver sin dificultad (bucles 2, 4 y 5), la vectorización ha sido muy eficiente, alcanzando *speedups* cercanos a un factor 4, lo que concuerda con el ancho del vector AVX2.

Por otro lado, en los bucles con dependencias de datos claras (1 y 3), el compilador ha sido conservador, evitando la vectorización automática para no alterar la semántica y los resultados del programa.

Si preguntamos a ChatGPT cómo actúa ICC en comparación con GCC y Clang, nos dice que el comportamiento de ICC es más analítico y conservador, emitiendo informes más detallados sobre las dependencias y costes estimados. Clang, por el contrario, tiende a aplicar heurísticas más agresivas y suele vectorizar incluso en casos donde la ganancia es dudosa. GCC se sitúa en un punto medio entre ambos, ofreciendo una buena estabilidad pero siendo algo más limitado en la detección automática de reducciones o reordenamientos de bucles.

4. Mediciones de paralelización en términos de tiempo.

Tenemos dos programas que implementan la integración numérica de la función $f(x) = \frac{1}{\sqrt{x}}$ en el intervalo $[0, 1]$, usando el método del punto medio. Aunque ambos calculan la misma integral, lo hacen empleando distintas técnicas de paralelización. En el caso de *integral-reduc.cpp*, se usa reducción (paralelización por datos), donde cada hilo calcula una parte de la integral y posteriormente se combinan todos los resultados. Por su parte, *integral-tasks.cpp* aplica una paralelización basada en recursividad (paralelización por tareas), donde el intervalo de integración se va dividiendo recursivamente en tareas más pequeñas y sencillas, para ejecutarse en paralelo con las demás y finalmente combinar los resultados parciales y obtener el de la integral.

- *Integral-reduc.cpp*:

	GCC	ICC	Clang
Tiempo (s)	1.4996	1.5225	1.4975
Resultado	1.99999395	1.99999395	1.99999395

- *Integral-tasks.cpp*:

	GCC	ICC	Clang
Tiempo (s)	14.9443	1.6126	1.6992
Resultado	1.99999395	1.99999395	1.99999395

Atendiendo a los resultados de la integral, comprobamos que ambos programas producen un resultado muy cercano a 2 (resultado esperado), con un error de 0.00000605. En cuestiones de tiempo, el programa que usa reducción tiene un rendimiento muy similar en los tres compiladores, alrededor de 1.5 segundos, lo que indica un uso eficiente del paralelismo. Por su parte, la versión *tasks* sube ligeramente los tiempos en ICC y Clang, apenas una décima de segundo, pero en GCC aumenta de manera muy considerable, con casi 15 segundos de ejecución.

Vamos a intentar analizar por qué se produce ese incremento tan grande en GCC. Una hipótesis puede ser que la implementación de OpenMP en GCC no optimiza adecuadamente esa técnica recursiva de división en tareas, ya que cada llamada recursiva genera nuevas tareas que el sistema de OpenMP tiene que organizar y repartir entre los hilos disponibles. Este proceso puede llegar a ser bastante costoso, debido a que el compilador debe coordinar las tareas, esperar a que terminen y combinar sus resultados. Si esa gestión no se hace de forma eficiente, el tiempo que se gasta en organizar las tareas puede ser incluso mayor que el tiempo que se tarda en hacer los cálculos, lo que provoca que el programa tarde mucho más en ejecutarse.

5. Benchmark *heat.cpp*.

5.1. Análisis del código.

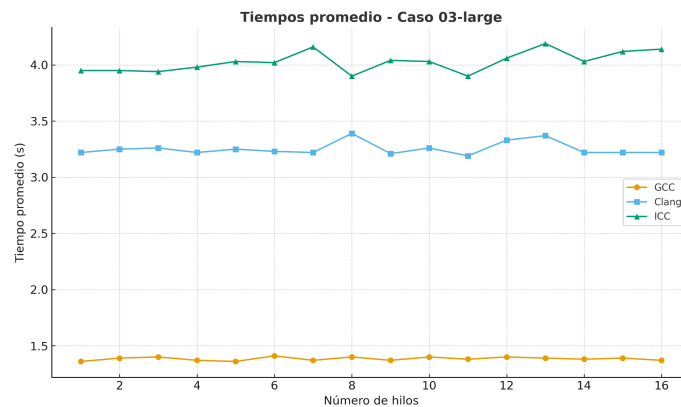
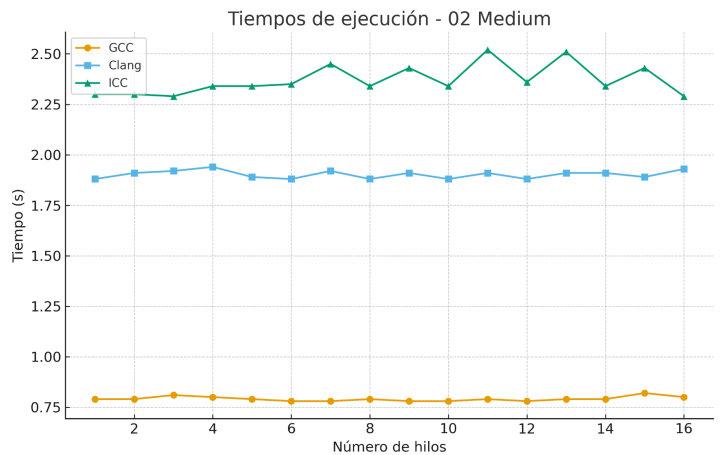
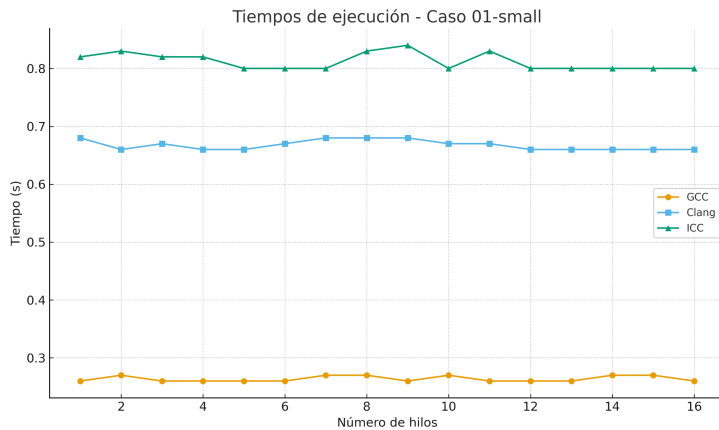
El programa *heat.cpp* tiene una única función llamada *solve*, la cual tiene 3 variables, una de tipo matriz, otra de tipo entero y otra de tipo *double*, y en su cuerpo tiene 3 bucles, uno grande de tipo *do while*, y dentro de él dos de tipo *for* anidados para recorrer la matriz.

Veamos si los bucles son candidatos para la paralelización:

- **Bucle 1 (*do while*):** es el bucle más grande de la función, se puede ver como un *for(difference=0; difference / (state.height * state.width) > tolerance; ++iterations)*. En cada iteración se calcula una nueva distribución de temperaturas en toda la matriz y se compara con la anterior. Este proceso depende directamente de los valores obtenidos en la iteración previa, ya que el nuevo estado se construye a partir del anterior. Por lo tanto, no existe independencia entre iteraciones, y el orden de ejecución es importante, así que este bucle no puede paralelizarse con OpenMP, ya que cada paso requiere los resultados del anterior.
- **Bucle 2 (*for height*):** este bucle es el encargado de recorrer las filas de la matriz. Para cada fila, se calcula la temperatura de cada punto interior combinando los valores de sus vecinos (arriba, abajo, izquierda y derecha), siguiendo el esquema *stencil* del método de propagación del calor. Dado que cada iteración del bucle trabaja sobre una fila distinta de la matriz y las filas son independientes entre sí, no existen dependencias de datos entre las iteraciones. Además, las operaciones leen de la matriz *state* y escriben en posiciones diferentes de otra matriz llamada *next_state*. Por tanto, podemos concluir que este bucle sí es candidato a paralelización mediante OpenMP. El pragma que habría que añadir es *#pragma omp parallel for*, que se encarga de distribuir las iteraciones del bucle en diferentes hilos.
- **Bucle 3 (*for width*):** este último bucle recorre las columnas de la matriz dentro de una misma fila. Ocurre como en el anterior, sus iteraciones son independientes, ya que cada elemento de *next_state* se calcula de forma separada y simplemente se necesita *state* para leer, por lo tanto este bucle sí es paralelizable. En teoría, también se podría aplicar un *#pragma omp parallel for* justo antes del bucle *j*, pero hacerlo dentro del bucle *i* generaría una región paralela por cada fila, lo que provocaría un coste de creación y sincronización de hilos en cada iteración. Después veremos en la práctica si es eficiente o realmente no merece la pena.

5.2. Análisis experimental.

En primer lugar, probamos a ejecutar sin ningún pragma, el código tal cual nos lo dan:



Observamos una tendencia constante en los tres casos, con GCC funcionando notablemente más rápido que los otros compiladores. ICC es el que más tiempo tarda y el que más variabilidad tiene, y Clang se acerca más en rendimiento a ICC, pero es ligeramente más rápido.

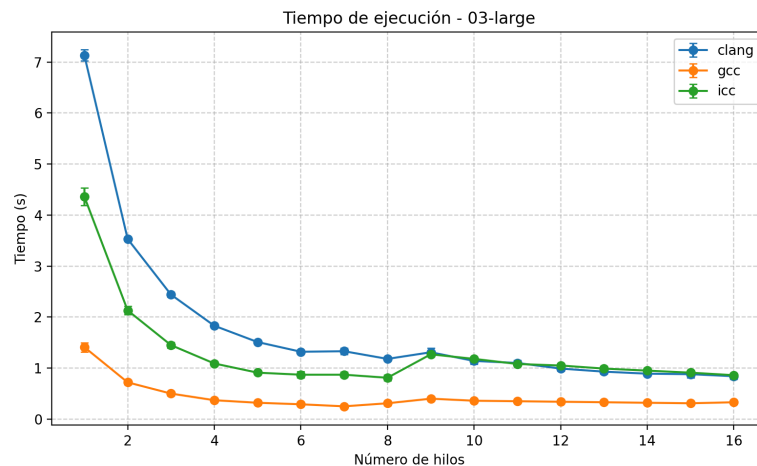
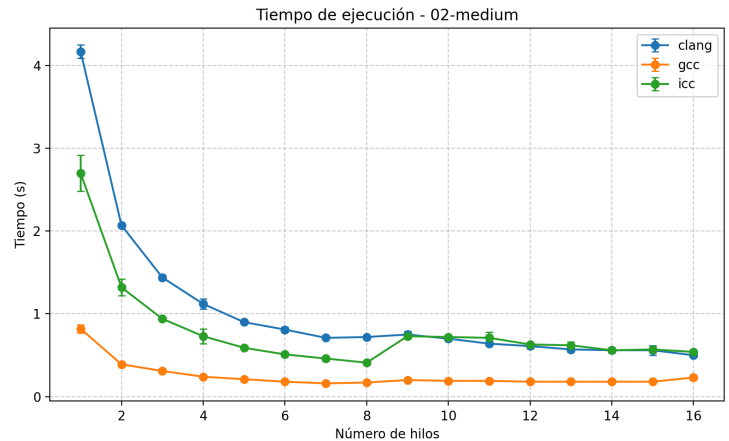
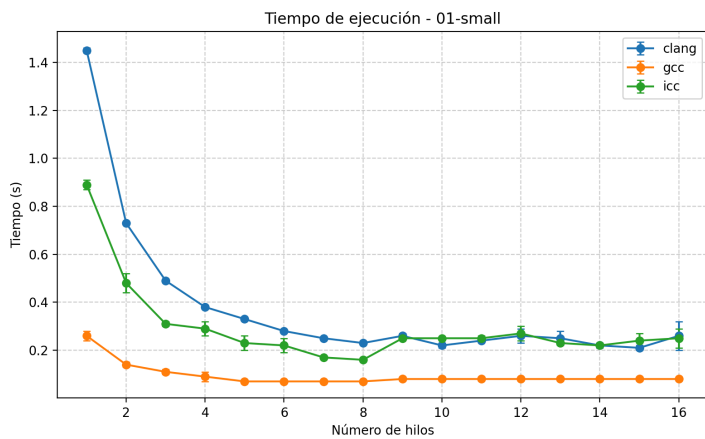
Vamos a comenzar probando a paralelizar el primer *for* (*height*). Si colocamos `#pragma omp parallel for` delante del bucle, cuando probamos *make-tests* obtenemos que para un hilo si da el resultado esperado pero para el resto no. ¿A qué se debe esto? Bien, es cierto que este bucle es paralelizable, pero si lo hacemos solo con ese pragma, la variable *difference* será modificada por todos los hilos al mismo tiempo, y provocando que no se obtenga el resultado esperado. Hay que aplicar la técnica de reducción, con este pragma: `#pragma omp parallel for reduction(+:difference)`. Así indicamos al compilador que cada hilo debe mantener su propia copia privada de *difference*, y al final de la paralelización, se sumarán todas en la variable final, evitando así conflictos entre hilos:

```

difference = 0;
#pragma omp parallel for reduction(+:difference)
for (size_t i = 1; i < state.height - 1; ++i) {
    for (size_t j = 1; j < state.width - 1; ++j) {

```

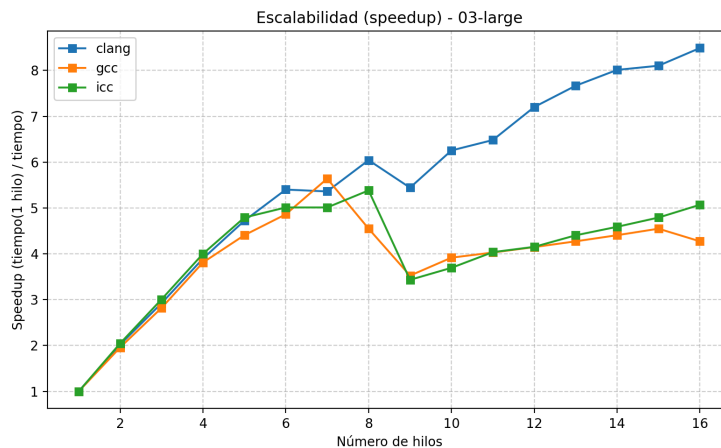
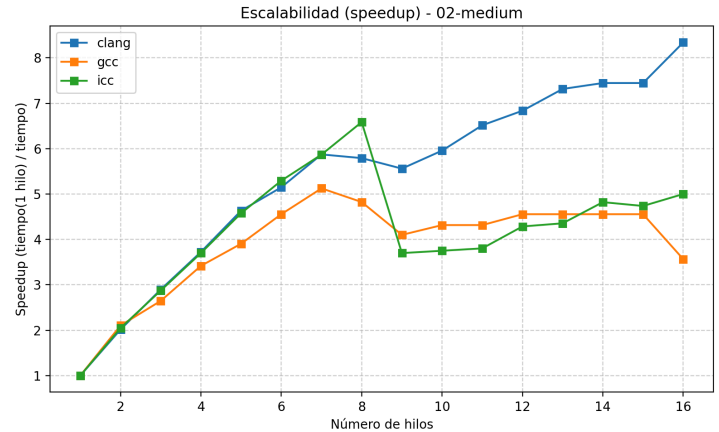
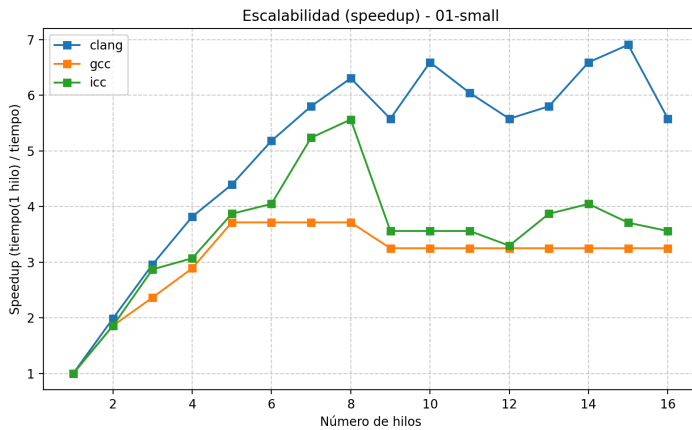
Ahora sí obtenemos los resultados esperados. Vamos a echar un vistazo a los tiempos de ejecución:



Observamos claramente el efecto de la paralelización, mejorando en gran escala los tiempos de ICC y Clang conforme aumenta el número de hilos. El compilador GCC es el que menos nota la paralelización, pues a partir de los 4-5 hilos el tiempo se mantiene estable. En cuestión de números, podemos comparar los tiempos para 16 hilos, en su versión 02 (medium), con respecto al programa sin paralelizar:

Compilador	Tiempo (sin paralelización)	Tiempo (con paralelización)
GCC	0.80	0.23
ICC	2.29	0.54
Clang	1.93	0.50

Podemos ver también de manera gráfica la escalabilidad del programa, con el speedup obtenido en función del número de hilos utilizados. Esta medida es muy útil, pues mide cuánto se acelera la ejecución del programa al aumentar el número de hilos respecto a la ejecución con un solo hilo.



Comparando los compiladores, Clang muestra la mejor escalabilidad global creciendo de forma estable hasta los 16 hilos, realizando un uso eficiente de los recursos de paralelismo y una buena gestión del reparto de trabajo entre hilos. En el caso de GCC, obtiene una escalabilidad moderada, con una mejora apreciable hasta aproximadamente 4–6 hilos, a partir de los cuales la ganancia se estabiliza. Esto indica que el rendimiento se ve limitado por la sobrecarga de sincronización o el ancho de banda de memoria. Por último, ICC se encuentra a caballo entre los dos, con buena escalabilidad inicial, especialmente en el caso 01 (*small*), pero un estancamiento posterior parecido al de *gcc*. En tamaños mayores, su rendimiento es más estable, aunque sin alcanzar el nivel de *clang*.

Si comparamos los tamaños del problema, en el más pequeño no se termina de explotar bien el paralelismo, ya que al ser reducido el tamaño, la creación y gestión de hilos no resulta tan positiva con respecto a todo el trabajo que realiza el programa. En los tamaños medio y grande la escalabilidad mejora notablemente, especialmente con *clang*, ya que el coste de la paralelización se amortiza y el trabajo por hilo es mayor, aprovechando mejor los recursos del sistema.

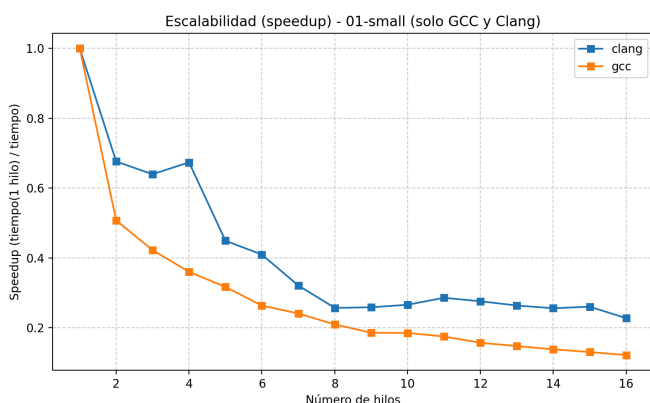
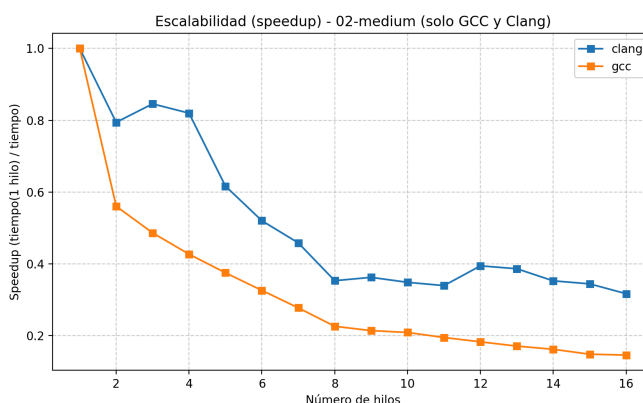
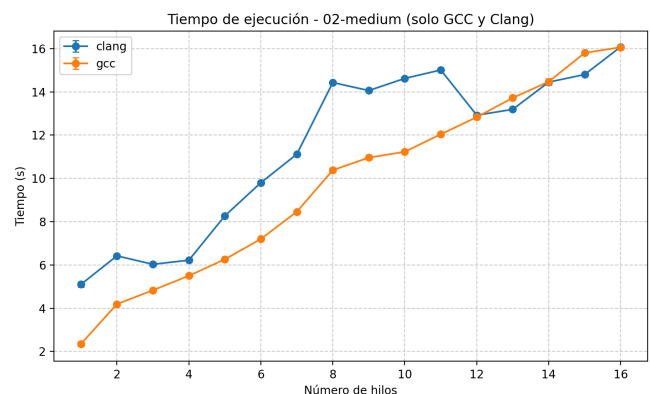
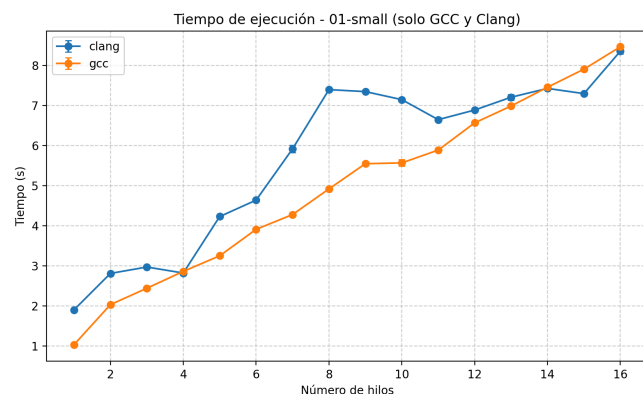
Vamos ahora a probar la paralelización en el segundo bucle. Para ello, solo tenemos que cambiar el pragma de línea y colocarlo justo antes del bucle de j:

```
for (size_t i = 1; i < state.height - 1; ++i) {
    #pragma omp parallel for reduction(+:difference)
    for (size_t j = 1; j < state.width - 1; ++j) {
        next_state[i][j] = (state[i][j]
                             + state[i + 1][j]
                             + state[i - 1][j]
                             + state[i][j + 1]
                             + state[i][j - 1]) / 5;
        difference = difference + abs(next_state[i][j] - state[i][j]);
    }
}
```

Si hacemos *make tests-all* obtenemos OK en todos los resultados, tal y como esperábamos. Sin embargo, cuando hacemos *make times-all*, obtenemos unos tiempos de ejecución demasiado lentos. He tenido que parar la ejecución en el tercer nivel de Clang, porque los tiempos eran hasta más lentos que sin paralelización. Lo vemos por ejemplo con 16 hilos en el nivel 2 de GCC y Clang:

Compilador	Tiempo (sin paralelización)	Tiempo (con paralelización 1)	Tiempo (con paralelización 2)
GCC	0.80	0.23	16.06
Clang	1.93	0.50	16.08

Vemos también las gráficas tanto de tiempo como de escalabilidad:



¿Por qué sucede esto si estamos paralelizando? Bien, al colocar el pragma dentro del bucle `i` provoca que en cada iteración se cree un grupo de hilos, se trabaje para sincronizarlos y al finalizar la iteración destruirlos. Es decir, el programa paraleliza muchas veces una tarea demasiado pequeña, lo que aumenta el tiempo total en lugar de reducirlo. Además, también se pierde eficiencia de caché, ya que los hilos acceden constantemente a datos contiguos que cambian entre filas, no favoreciendo así el aprovechamiento de la localidad espacial.

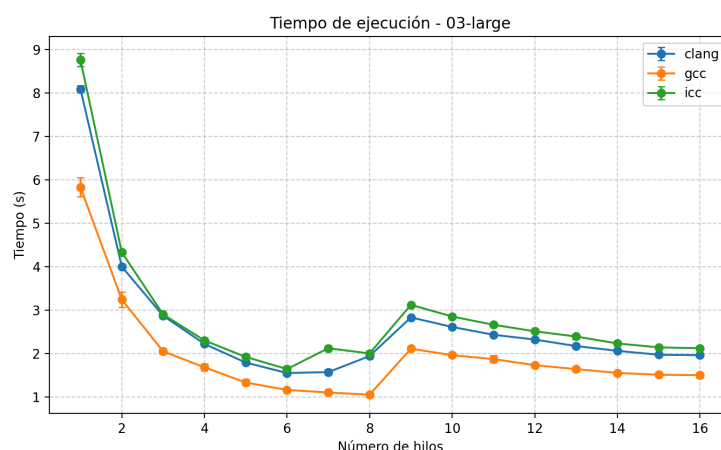
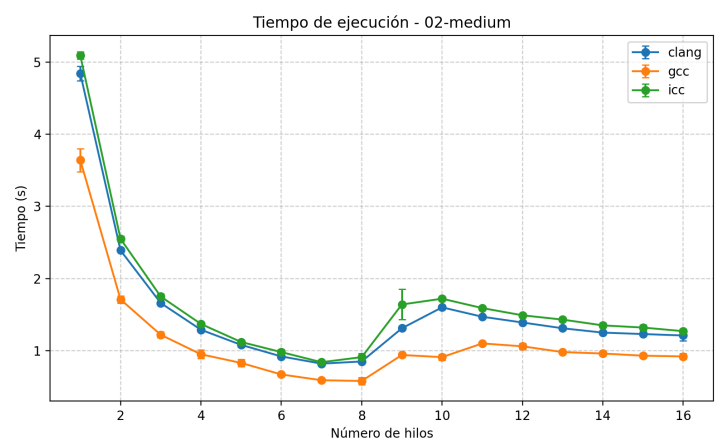
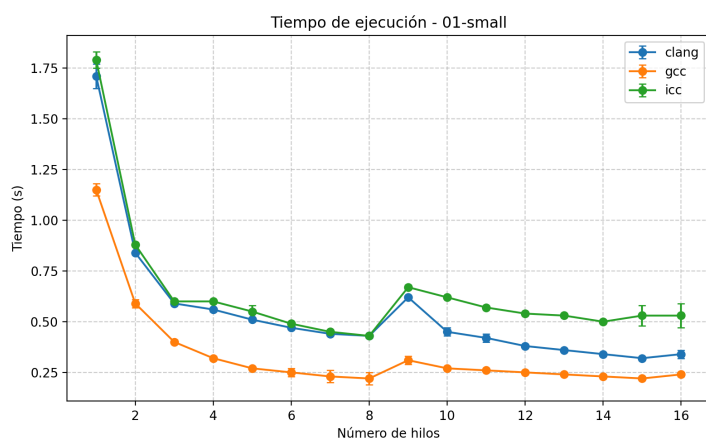
Nuestra última prueba de paralelización será paralelizar ambos bucles a la vez. Para ello, lo haremos a través de la cláusula `collapse(2)` junto con `#pragma omp parallel for`. Según la documentación oficial de OpenMP, dice para la cláusula `collapse(n)` lo siguiente: “a constant positive integer expression that specifies how many loops are affected by the construct”. Es decir, `collapse` combina bucles anidados para ser tratados como un único bucle en la distribución de trabajo, y `n` es un entero positivo constante que indica cuántos bucles anidados se combinan.

En nuestro caso, al utilizar `collapse(2)`, OpenMP combina las iteraciones de los dos bucles anidados (`i` y `j`) en una sola secuencia lineal de iteraciones. De esta manera, el conjunto total de iteraciones se reparte de forma más uniforme entre los hilos, aprovechando mejor el paralelismo de datos disponible en toda la matriz.

```
#pragma omp parallel for collapse(2) reduction(+:difference)
```

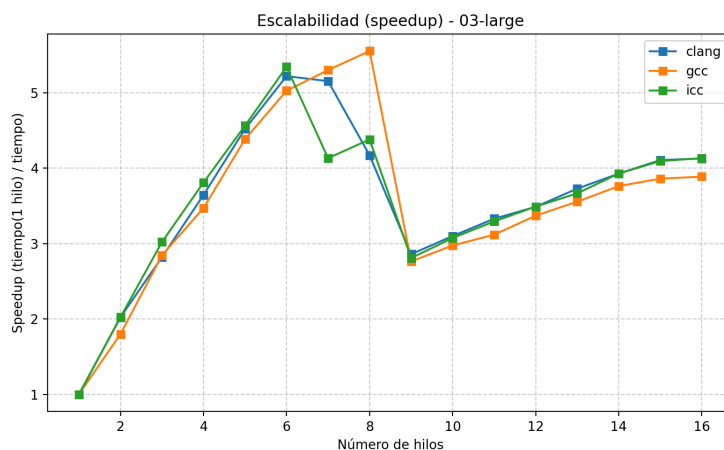
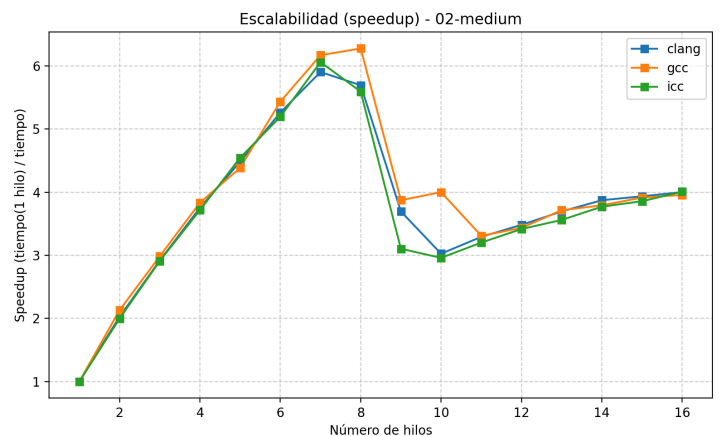
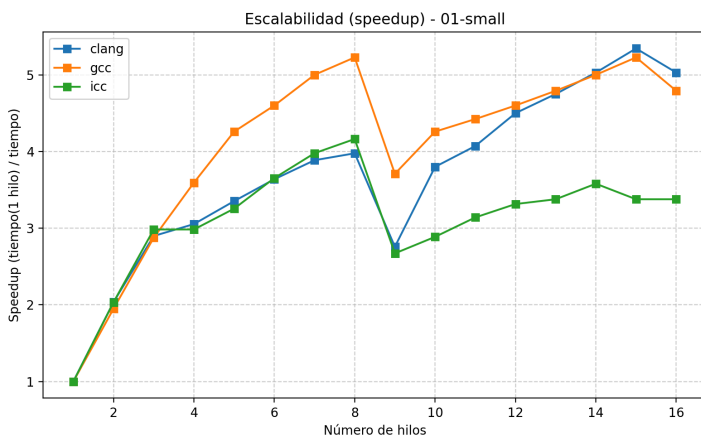
```
for (size_t i = 1; i < state.height - 1; ++i) {  
    for (size_t j = 1; j < state.width - 1; ++j) {
```

Vamos a ver los tiempos de ejecución de nuevo gráficamente:



El comportamiento de la paralelización es muy similar al de la primera paralelización (solo el primer for), se puede ver la tendencia descendente de tiempo conforme aumenta el número de hilos, aunque sin embargo, el repunte cuando se alcanzan los 8 hilos es más grande en este caso.

Podemos ver también la escalabilidad con esta opción de paralelización. De esta manera comprobaremos si el paralelismo es realmente útil, tal y como sucedía en las gráficas de escalabilidad de la mejor paralelización hasta ahora, la del bucle i.



Podemos apreciar un comportamiento similar al obtenido cuando solo se paraleliza el bucle externo (i), aunque con una eficiencia algo menor a partir de los 8 hilos. En general, el *speedup* crece de forma casi lineal hasta los 6-8 hilos, donde se produce una caída del rendimiento, probablemente debida al incremento del coste de sincronización y la gestión de memoria compartida. En comparación con la paralelización del primer bucle, esta versión presenta una escalabilidad más irregular y menos estable.

La diferencia más significativa se presenta en los tiempos, si comparamos con las otras ejecuciones (sin paralelización y con paralelización del bucle i), para 8 hilos, en todas las versiones:

Versión 01-small:

Compilador	Tiempo (sin paralelización)	Tiempo (con paralelización 1)	Tiempo (paral. collapse)
GCC	0.27	0.07	0.22
ICC	0.83	0.16	0.43
Clang	0.68	0.23	0.43

Versión 02-medium:

Compilador	Tiempo (sin paralelización)	Tiempo (con paralelización 1)	Tiempo (paral. collapse)
GCC	0.79	0.17	0.58
ICC	2.34	0.41	0.91
Clang	1.88	0.72	0.85

Versión 03-large:

Compilador	Tiempo (sin paralelización)	Tiempo (con paralelización 1)	Tiempo (paral. collapse)
GCC	1.40	0.31	1.05
ICC	3.90	0.81	2.00
Clang	3.39	1.18	1.94

Con todos estos datos, podemos concluir que la paralelización del primer bucle (i) es la más eficiente de todas las versiones probadas. Para todos los tamaños del problema, esta opción nos da los mejores tiempos de ejecución y una mejor relación entre el número de hilos y la reducción de tiempo, mostrando una escalabilidad más estable. Sin embargo, la versión con paralelización mediante *collapse* (ambos bucles a la vez) presenta un rendimiento notablemente inferior, incluso llegando a ser más lenta que la versión sin paralelización en algunos casos. Esto se debe al mayor coste de sincronización y de gestión de hilos generada por el doble nivel de paralelismo. En definitiva, la paralelización del bucle externo proporciona el mejor equilibrio entre eficiencia, escalabilidad y estabilidad del rendimiento.